RV UNIVERSITY

School of Computer Science and Engineering

WWLLM LABORATORY – CIE-3

MINI-PROJECT REPORT

DASTAVEZ: INDIAN DOCUMENT UNDERSTANDING TOOL

Reading, extracting, summarising and translating Indian-language documents using Indian AI models

Submitted by

Pranav Nayak

1RVU23CSE344

November 2026

ABSTRACT

Notices, fee slips, admit cards and bills are routinely issued in the regional languages of India, and recipients who do not read the language of a document must depend on others to interpret it. Errors in reading an amount, a date or an identification number can lead to financial loss or missed deadlines. This report presents Dastavez, a web-based Indian Document Understanding Tool that accepts a photograph, scan or PDF of such a document and produces its text, its key fields, a plain-English summary, a translation into any of eleven languages, and answers to questions about its content.

All artificial-intelligence models used in the system are Indian-built models provided by Sarvam AI: Sarvam Vision 2.1 for optical character recognition and field extraction, Sarvam-105B for summarisation and question answering, and Sarvam-Translate v1 for translation. The application is implemented in Python with the Flask framework and a custom interface written in HTML, CSS and JavaScript.

The system was evaluated on 58 test cases derived from 40 synthetic documents in Hindi, Kannada and Tamil, for which the correct answers are known exactly. On clean scans it achieved a field-extraction accuracy of 100% (160 of 160 fields) and a question-answering accuracy of 100% (40 of 40 questions). On copies degraded to resemble blurred mobile-phone photographs, the corresponding accuracies were 83.3% (60 of 72) and 88.9% (16 of 18); every error occurred at the most severe degradation level and was caused mainly by the misreading of individual digits. The report also describes the engineering problems encountered, the design decisions taken in response, and the deviations from the project brief.

Keywords: document understanding, optical character recognition, Indian languages, Sarvam AI, machine translation, question answering, large language models.

TABLE OF CONTENTS

Chapter 1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 2
1.5 Organisation of the Report 2

Chapter 2 Technologies and Models Used **3**
2.1 Indian AI Models Used 3
2.2 Models Named in the Brief That Were Investigated 3
2.3 Software and Tools 4
2.4 Use of AI-Assisted Development 4

Chapter 3 System Design **5**
3.1 System Architecture 5
3.2 Processing Pipeline 5
3.3 Module Description 6
3.4 Interface Design 6
3.5 Design Decisions 7

Chapter 4 Implementation **8**
4.1 Document Reading and Field Extraction 8
4.2 Summarisation and Question Answering 8
4.3 Document Translation 8
4.4 User Interface 9
4.5 Robustness and Security 15

Chapter 5 Testing and Results **16**
5.1 Test Sets 16
5.2 Evaluation Metrics 17
5.3 Results 17
5.4 Failure Analysis 18
5.5 Discussion 19

Chapter 6 Challenges Encountered and Solutions **20**

Chapter 7 Deviations from the Project Brief **22**

Chapter 8 Conclusion and Future Scope **23**
8.1 Conclusion 23
8.2 Limitations 23
8.3 Future Scope 23

References **25**

Appendix A: Installation and Execution **26**

LIST OF FIGURES

Figure 3.1: System architecture of Dastavez 5
Figure 4.1: Landing page of the Dastavez website 10
Figure 4.2: Upload screen with the document preview and the language selector 11
Figure 4.3: One-click sample documents in Hindi, Kannada and Tamil 11
Figure 4.4: Live progress while the document is being read 12
Figure 4.5: Extracted fields and English summary of a Hindi fee notice 12
Figure 4.6: The same document translated into Kannada, shown beside the original 13
Figure 4.7: Question answering on the document 13
Figure 4.8: Layout on a phone-sized screen 14
Figure 4.9: The “How it works” section of the website 15
Figure 5.1: The same layout at each severity level (Tamil fee notice) 16
Figure 5.2: Results section of the website, with accuracy against photograph quality 18

LIST OF TABLES

Table 1.1: Requirements of the project brief and how they were met 2
Table 2.1: Indian AI models used in the system 3
Table 2.2: Models named in the brief and the outcome of the investigation 3
Table 2.3: Software and tools used 4
Table 3.1: Source files of the project 6
Table 3.2: Endpoints of the Flask server 6
Table 3.3: Sarvam AI API calls used 7
Table 5.1: Composition of the test sets 16
Table 5.2: Accuracy on the clean and phone-photograph test sets 17
Table 5.3: Field accuracy by field on the phone-photograph set 18
Table 5.4: Failure modes observed on the harsh phone-photograph documents 19
Table 6.1: Problems encountered and the solutions adopted 20
Table 7.1: Deviations from the project brief 22

CHAPTER 1
INTRODUCTION

1.1 Background

India has twenty-two scheduled languages and many more regional ones, and official and institutional documents are frequently issued in the language of the state or institution concerned. A student relocating to another state, or a parent who reads a different language from the one used by the school, may receive a fee notice or an examination admit card that cannot be read with confidence. Interpreting such a document requires help from another person, and a mistake in an amount, a date or a roll number can have real consequences.

Recent progress in multilingual document-reading models and large language models makes it possible to automate much of this task. Several Indian organisations, including AI4Bharat, BharatGen and Sarvam AI, now provide models trained specifically for Indian languages and scripts. This project applies such models to the practical problem of understanding a document that the reader cannot easily read.

1.2 Problem Statement

The project brief (*Mini Projects using AI4Bharat & BharatGen*, CS3235) describes the project as follows: “Upload an Indian-language notice, form or government document; extract fields, summarize and answer document questions.” The aim is to build a usable, end-to-end application, based on pretrained Indian-language models, with measurable evaluation.

1.3 Objectives

1. To develop an end-to-end web application that accepts an image or PDF of an Indian-language document.
2. To use Indian-built artificial-intelligence models for reading, understanding and translating the document.
3. To extract key fields, generate a plain-English summary, translate the document into a language chosen by the user, and answer questions about it.
4. To evaluate the system quantitatively on 30 to 50 representative test cases and to analyse its failures.
5. To provide a clear and usable interface, together with documentation and source code.

Table 1.1: Requirements of the project brief and how they were met

Requirement	How it was met
Working end-to-end application with a usable frontend	A custom website (HTML, CSS, JavaScript) served by a Flask server; tested in a real browser and on a phone-sized screen.
Indian-language models and measurable evaluation	Sarvam Vision 2.1, Sarvam-105B and Sarvam-Translate v1; evaluation scripts, result files and result charts built into the website.
Generative-AI integration	Language-model summaries and document question answering; schema-driven field extraction.
30 to 50 test cases, failure analysis	58 test cases (40 clean and 18 degraded) with field-extraction and question-answering accuracy; four failure modes analysed from the raw model output (Chapter 5).
Demonstration, documentation and repository	Working demonstration with sample documents; README file and this report; complete source-code package.

1.4 Scope

The application accepts PNG, JPEG and PDF files in eleven languages (Hindi, Kannada, Tamil, Telugu, Marathi, Bengali, Gujarati, Malayalam, Punjabi, Odia and English). The evaluation covers Hindi, Kannada and Tamil documents of two types: fee-payment notices and examination admit cards. All test documents are synthetic. The limitations that follow from this scope are discussed in Chapter 8.

1.5 Organisation of the Report

Chapter 2 describes the models and tools used. Chapter 3 presents the system design. Chapter 4 describes the implementation, including the user interface with screenshots. Chapter 5 presents the testing methodology, the results and the failure analysis. Chapter 6 records the problems encountered and their solutions. Chapter 7 lists the deviations from the project brief. Chapter 8 gives the conclusion, the limitations and the scope for future work.

CHAPTER 2
TECHNOLOGIES AND MODELS USED

2.1 Indian AI Models Used

Three Indian-built models, all provided by Sarvam AI through its public application programming interface (API), form the artificial-intelligence core of the system. They are summarised in Table 2.1.

Table 2.1: Indian AI models used in the system

Model	Developer	Role in the system
Sarvam Vision 2.1 (Document AI: *parse* and *extract*)	Sarvam AI, India	Reads the page image or PDF using optical character recognition (OCR) for 22 Indian scripts and English, returns the text with tables preserved, and extracts named fields according to a JSON schema. Supports 23 languages.
Sarvam-105B (`sarvam-105b`)	Sarvam AI, India	A reasoning language model used to generate the English summary and to answer questions about the document. It replaced the earlier model `sarvam-m`, which was deprecated.
Sarvam-Translate v1 (`sarvam-translate:v1`)	Sarvam AI, India	Translates document text between Indian languages and English. It was selected after comparison with Sarvam's `mayura:v1` model on the same text (Section 4.3).

2.2 Models Named in the Brief That Were Investigated

The project brief names BharatGen Patram and AI4Bharat IndicTrans2 as candidate models. Both were investigated before the final choice of models was made; the findings are given in Table 2.2.

Table 2.2: Models named in the brief and the outcome of the investigation

Model	Finding
BharatGen Patram (7B vision-language model)	7.7 billion parameters and 15.3 GB of weights, with no hosted version available. It could not be run on the available 6 GB graphics card, and the measured network speed of about 0.1 to 0.4 MB/s made the download impractical.
AI4Bharat IndicTrans2	Access-gated on Hugging Face (the available account did have access); the smallest relevant variant is 0.9 GB. It was set aside together with Patram because hosted inference through Hugging Face required paid credits (HTTP status 402) and the official AI4Bharat IndicTrans3 demonstration space was returning a configuration error at the time.

2.3 Software and Tools

Table 2.3: Software and tools used

Tool	Purpose
Python 3.13	Implementation language for the server, pipeline, evaluation and test-set generation.
Flask 3.1, `requests`	Web server and JSON API; calls to the Sarvam REST API with retries, time-outs and readable error messages.
HTML5, CSS3, JavaScript (no frameworks)	The website: landing page, document desk, live progress, results ledger, tabs and charts. Typefaces: Fraunces, Instrument Sans, IBM Plex Mono and Noto Serif Devanagari.
pandas, NumPy, Pillow	Accuracy computation and result tables; generation of degraded (phone-photograph-like) test images.
python-dotenv	Loads the API key from a local, version-control-ignored `.env` file.
Microsoft Edge (headless)	Renders the synthetic test documents so that Indic conjunct characters are shaped correctly, captures screenshots, and prints this report to PDF.
Chrome DevTools Protocol	Drives a real browser for automated end-to-end testing of every screen and detection of JavaScript errors.
Hugging Face Hub	Inspection of model sizes, access and hosting during the model investigation.
Claude Code (Anthropic)	AI coding assistant used throughout the development (Section 2.4).

2.4 Use of AI-Assisted Development

One purpose of the project is to demonstrate how effectively AI tools can be used to build a complete application. Claude Code, an AI coding assistant from Anthropic, was used as a programming partner throughout the work, under the direction of the author. The assistant was used to compare the candidate project topics, to investigate model sizes and hosting options, to read the API documentation and test the endpoints before relying on them, to write the server, the pipeline, the evaluation code and the website, to generate the test sets, and to test the interface in a real browser.

Several first attempts made with this assistance were found to be wrong and were corrected after testing. The first scoring code counted API failures as wrong answers; the first translation approach dropped a line of the document; and one chart was stacked and incorrectly ordered. Each was fixed and re-verified. The topic, the requirement to use Indian models only, the preference for a purpose-designed interface and the review of all results were decisions of the author. Every figure quoted in this report is taken from the saved evaluation output in the `results` directory of the source package.

CHAPTER 3
SYSTEM DESIGN

3.1 System Architecture

The system has a three-tier structure, shown in Figure 3.1. The web browser communicates only with a Flask server running on the user's own computer. The server hands each request to the pipeline module, which uses an API client module to call the Sarvam AI models. The API key is held only by the server and is never sent to the browser.

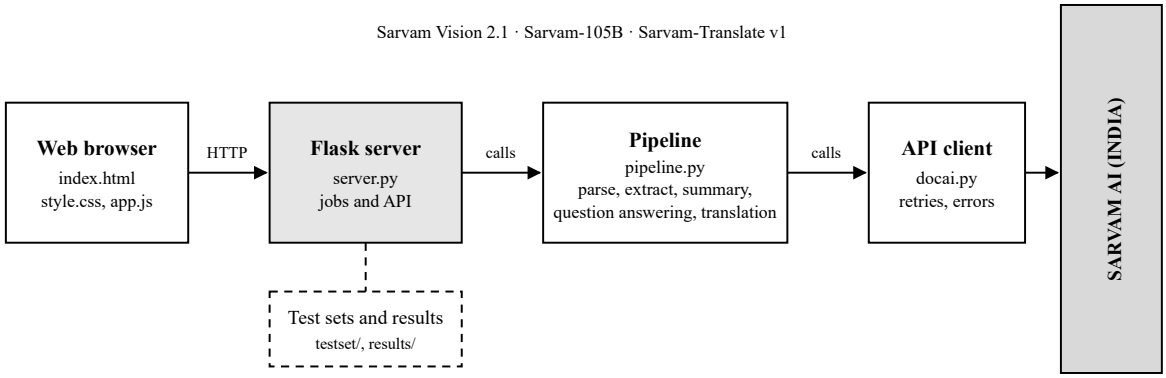


Figure 3.1: System architecture of Dastavez

3.2 Processing Pipeline

1. **Upload.** The browser sends the file (or the name of a sample document) and the selected language to the endpoint `POST /api/analyze`. The server validates the file type, the size (at most 25 MB) and the language, starts a background job and returns a job identifier.
2. **Reading and extraction in parallel.** Two requests run at the same time on Sarvam Vision: *parse*, which performs OCR and returns the page as Markdown with tables, and *extract*, which returns six named fields according to a JSON schema.
3. **Summarisation.** As soon as the text is available, Sarvam-105B generates the English summary while field extraction completes.
4. **Progress reporting.** The browser polls `GET /api/job/<id>` every 0.9 seconds and displays the completion of each of the three steps.
5. **Exploration.** Translation (`/api/translate`) and question answering (`/api/ask`) are performed on demand using the stored text of the document.

3.3 Module Description

Table 3.1: Source files of the project

File	Purpose
`server.py`	Flask server: serves the website, runs analysis jobs, provides the JSON API and the results endpoint.
`pipeline.py`	Field schema, summary and question prompts, row parsing, and cell-by-cell translation with number checks.
`docai.py`	Sarvam API client for parse, extract, chat and translate, with retries, time-outs and readable errors.
`web/index.html`, `style.css`, `app.js`	The website, written without external frameworks.
`evaluate.py`	Scores field extraction and question answering; resumable; skips API failures instead of scoring them.
`make_testset.py`, `make_hard_testset.py`	Generate the clean and the degraded test sets.
`testset/`, `testset_hard/`, `results/`	Test documents with ground truth, and the evaluation output (CSV and JSON files).
`README.md`, `run_app.bat`, `requirements.txt`, `.env.example`	Setup instructions and configuration template. The real `.env` file is excluded from the package.

3.4 Interface Design

Table 3.2: Endpoints of the Flask server

Endpoint	Purpose
`GET /api/samples`	Lists the six sample documents.
`POST /api/analyze`	Uploads a file (or selects a sample) with a language and starts a job.
`GET /api/job/<id>`	Returns the progress of the three steps and, when complete, the result.
`POST /api/translate`	Translates the stored document into a chosen language.
`POST /api/ask`	Answers a question from the stored document.
`GET /api/results`	Returns the evaluation numbers and mistakes shown in the results section of the website.

Table 3.3: Sarvam AI API calls used

Call	Used for
`POST /doc-ai/v1/job/parse`	OCR to Markdown (`output_format = md`); the result is fetched as a ZIP archive through the download-URL call.
`POST /doc-ai/v1/job/extract`	Schema-based field extraction (`output_format = json`); the result is fetched through the results call.
`POST /v1/chat/completions`	Model `sarvam-105b`: summary and question answering.
`POST /translate`	Model `sarvam-translate:v1`: translation, one table cell or line per call.

3.5 Design Decisions

1. **A purpose-built website instead of a dashboard framework.** A prototype built with Streamlit worked but looked generic and restricted the layout. The final interface uses plain HTML, CSS and JavaScript. All text received from the server is inserted using `textContent` and never as raw HTML, which prevents injection of markup from document contents.
2. **Background jobs with polling.** An analysis takes between 20 and 40 seconds. A job identifier with periodic polling allows the page to show real progress and to tolerate slow API responses, instead of a single request that appears frozen.
3. **Two independent readers of the same image.** OCR (*parse*) and field extraction (*extract*) are separate model calls. In one failure case the OCR output was hallucinated while the extraction still read the person's name correctly, so the two act as a cross-check.
4. **Answers only from the document.** The question prompt forbids the use of outside knowledge and requires the reply “Not found” when the answer is absent, to avoid confident invention.
5. **Key handling.** The API key is stored in a version-control-ignored `.env` file and is read only by `docai.py`.

CHAPTER 4
IMPLEMENTATION

4.1 Document Reading and Field Extraction

The *parse* operation of Sarvam Vision returns the text of the document as Markdown, with tables expressed as HTML table markup. The *extract* operation receives a JSON schema describing six fields (document type, organisation, person's name, identification number, date and amount) together with a short description of each, and returns their values as JSON. Both operations are asynchronous: a job is submitted, its status is polled every three seconds until it reaches a terminal state, and the output is then fetched.

On one blurred photograph the parser returned an entire region of the page as an embedded base64 image instead of text, which would have consumed a very large number of language-model tokens. The parsed text is therefore post-processed to replace such image blobs with a short placeholder before it is used.

4.2 Summarisation and Question Answering

Summaries and answers are generated by Sarvam-105B through the chat-completions API. Four points were found necessary after testing. First, the model is a reasoning model that spends roughly one thousand tokens on internal reasoning before answering, so the token budget was raised to 4,000, with one retry at double the budget if a reply is cut off. Second, the summary prompt instructs the model to write in English whatever the language of the document, because a summary of a Hindi document was once returned in Hindi. Third, the prompt forbids guessing a person's gender from a first name. Fourth, the question prompt restricts the model to the document text and requires the reply "Not found" when the answer is not present.

4.3 Document Translation

Translation was the most carefully engineered part of the system, because direct translation of a full page into Kannada or Tamil produced unacceptable results: lines were merged, one line was dropped, the table markup was corrupted, and in Tamil the date `02/02/2026` was converted into "October 2, 2026". The final design has four parts.

1. **Cell-by-cell translation.** The parsed text is divided into table cells and lines, and each is translated by a separate request, so that no content can be merged or dropped.
2. **Protected values.** Any cell consisting of ASCII characters that contains a digit (identification numbers, dates, amounts, reference numbers) is copied exactly as written and is never sent to the translator.

3. **Number verification.** After translation, the digits in every cell are compared with those of the original (native-script digits are converted first). If they differ, the cell is flagged with a warning symbol in the interface.
4. **Long text.** Lines longer than 900 characters are split at sentence boundaries, because the API accepts a limited number of characters per request and an earlier version silently truncated longer lines.

Two translation models were compared on the same cells. The older `mayura:v1` rendered “roll number” as “cylinder number” in Tamil and “fee” as “fare” in English, and inserted a Telugu word into the Kannada output. `sarvam-translate:v1` corrected all three and was therefore adopted. Because individual API calls take between one and seven seconds, the cells of a document are translated in parallel, one worker per cell, which reduced the time for a whole document from 22 seconds (four workers) or 53 seconds (one at a time) to between one and three seconds when the service is responsive.

4.4 User Interface

The interface is a single web page that combines the landing section, the working tool, the evaluation results and an explanation of the processing steps. The screens are described below using screenshots of the running application.

4.4.1 Landing Page

Figure 4.1 shows the landing page. It presents the purpose of the tool, the supported scripts, and a stack of sample documents with the fields that the system identifies.

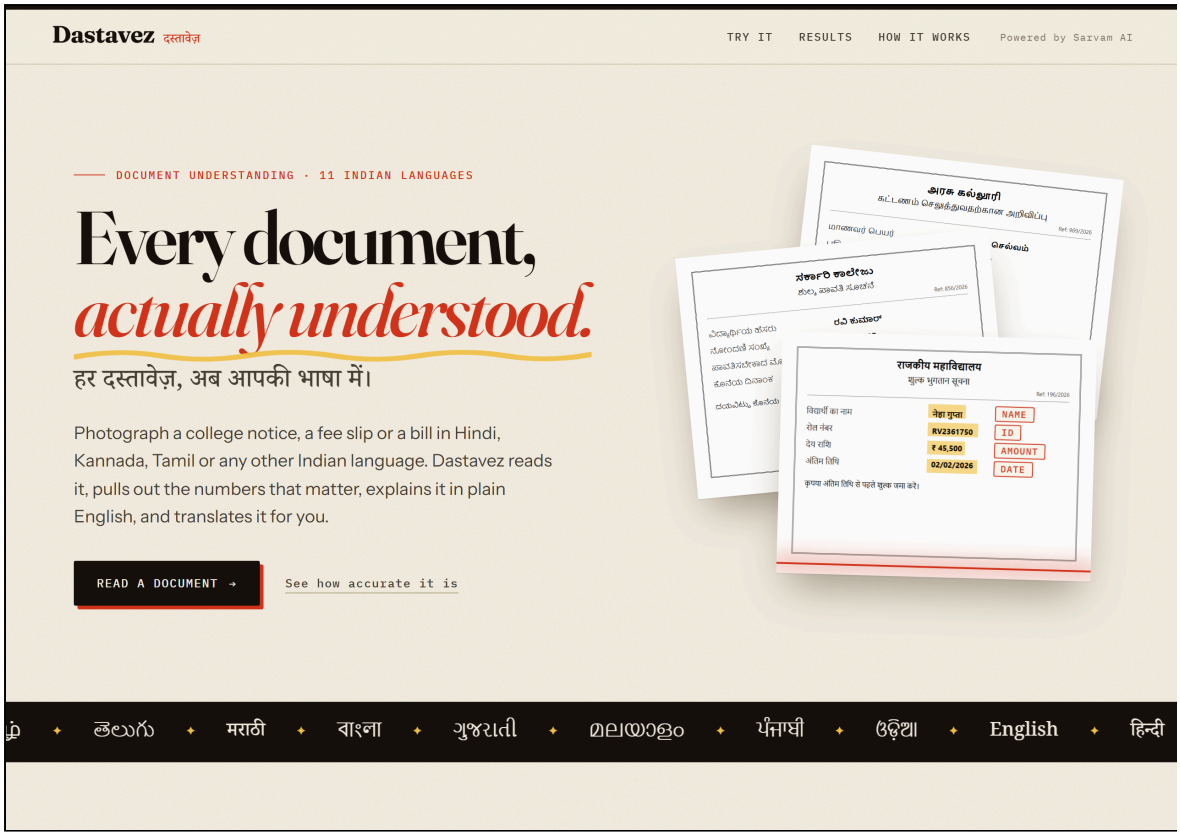


Figure 4.1: Landing page of the Dastavez website

4.4.2 Selecting a Document

A document may be supplied by dragging a file onto the upload area (Figure 4.2) or by choosing one of six sample documents (Figure 4.3). The language of the document is selected below the preview; samples set it automatically.

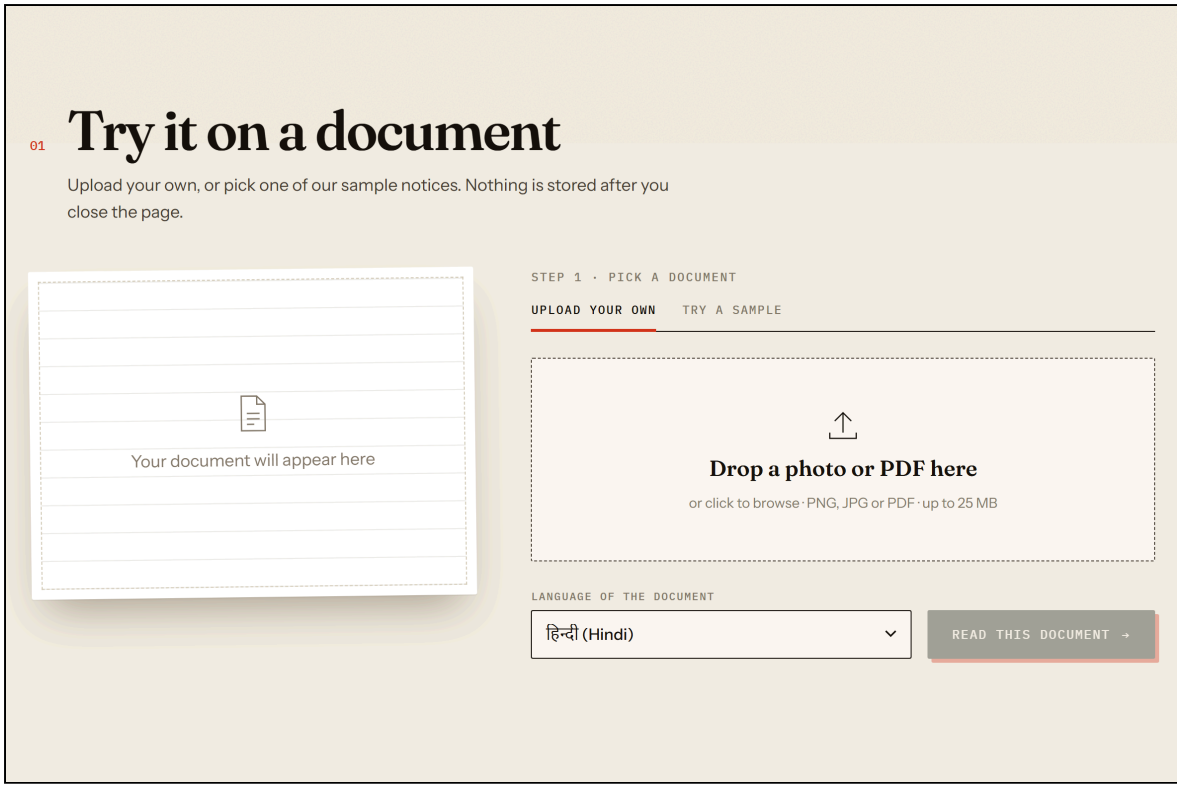


Figure 4.2: Upload screen with the document preview and the language selector

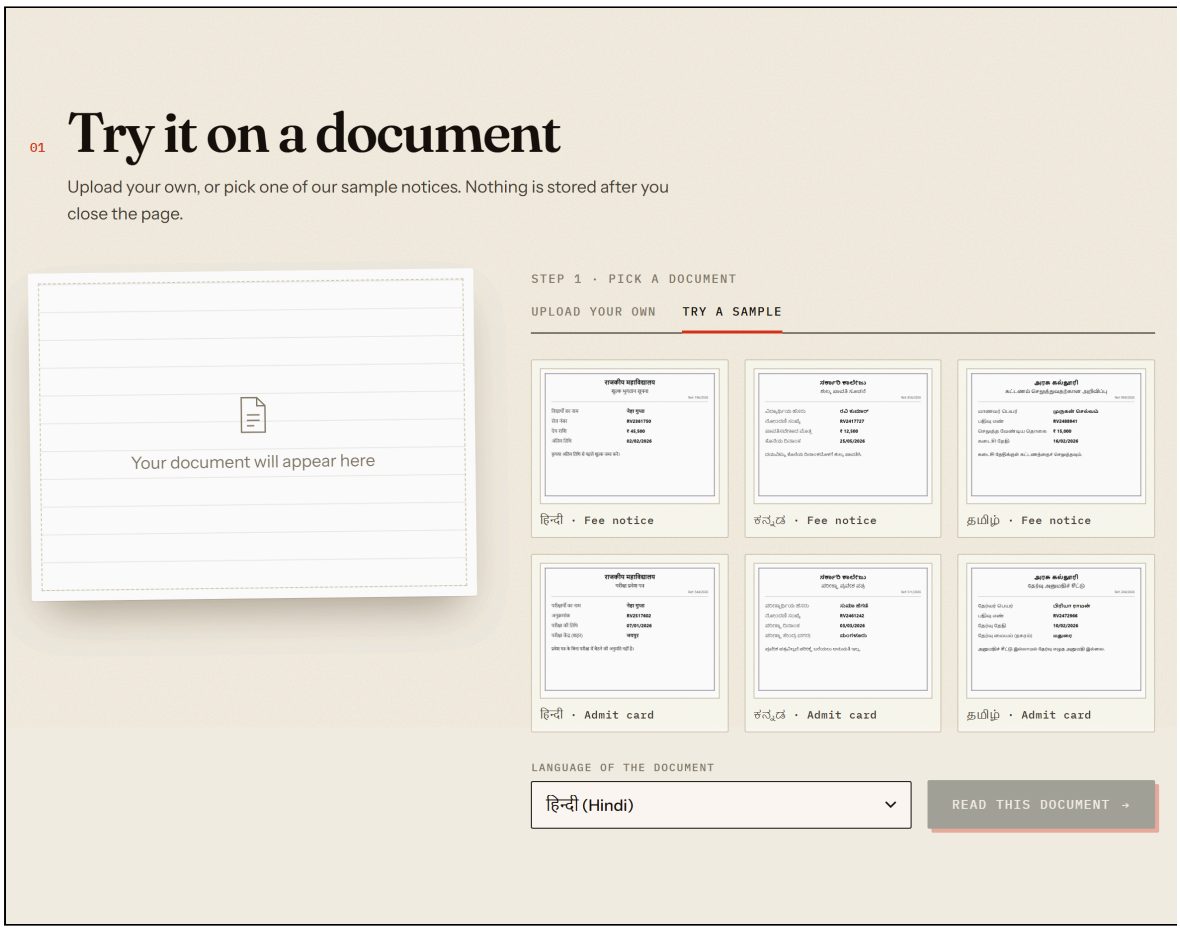


Figure 4.3: One-click sample documents in Hindi, Kannada and Tamil

4.4.3 Reading Progress

While the models are working, a scan line moves over the document and the three steps are marked as they complete (Figure 4.4).

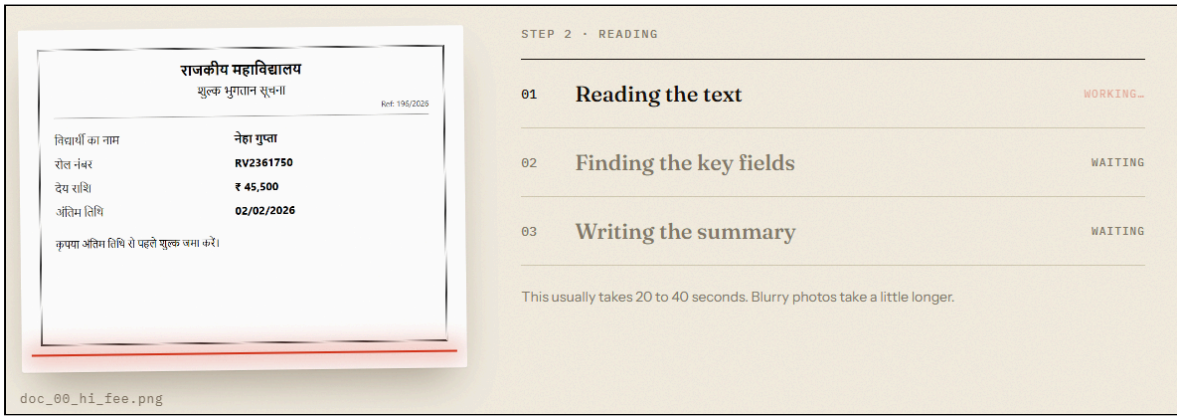


Figure 4.4: Live progress while the document is being read

4.4.4 Results

The results (Figure 4.5) consist of the document type and issuing organisation as a heading, the extracted fields in a ledger, and the English summary. The preview of the original document remains beside the results so that every value can be compared with the source.

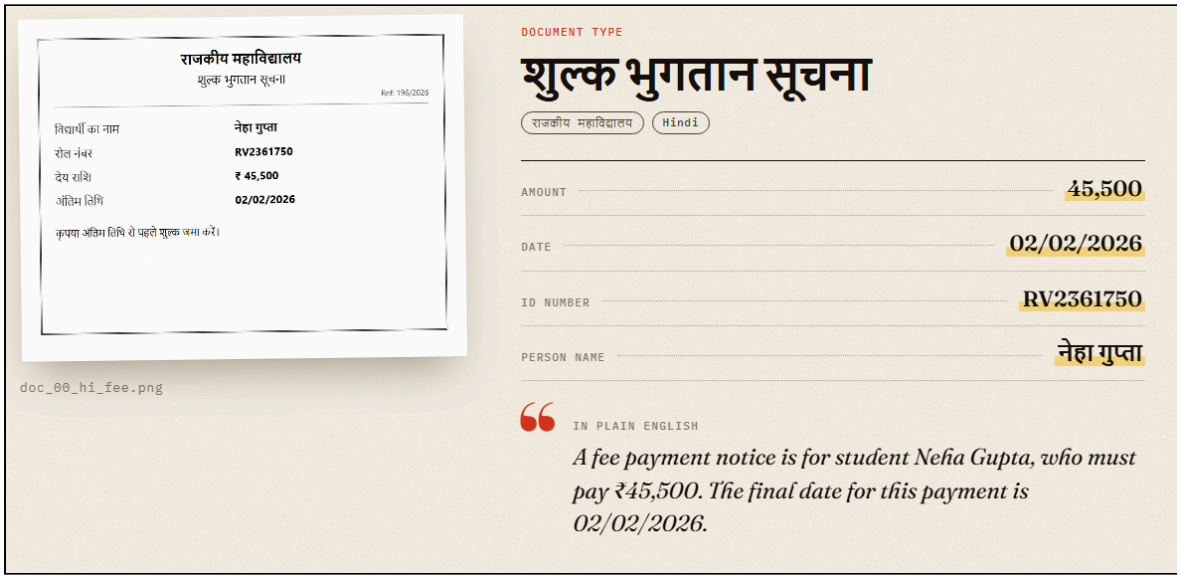


Figure 4.5: Extracted fields and English summary of a Hindi fee notice

4.4.5 Translation

The translation tab (Figure 4.6) shows the original text beside the translation. Identification numbers, dates and amounts are reproduced exactly. The translation can be downloaded as a text file.

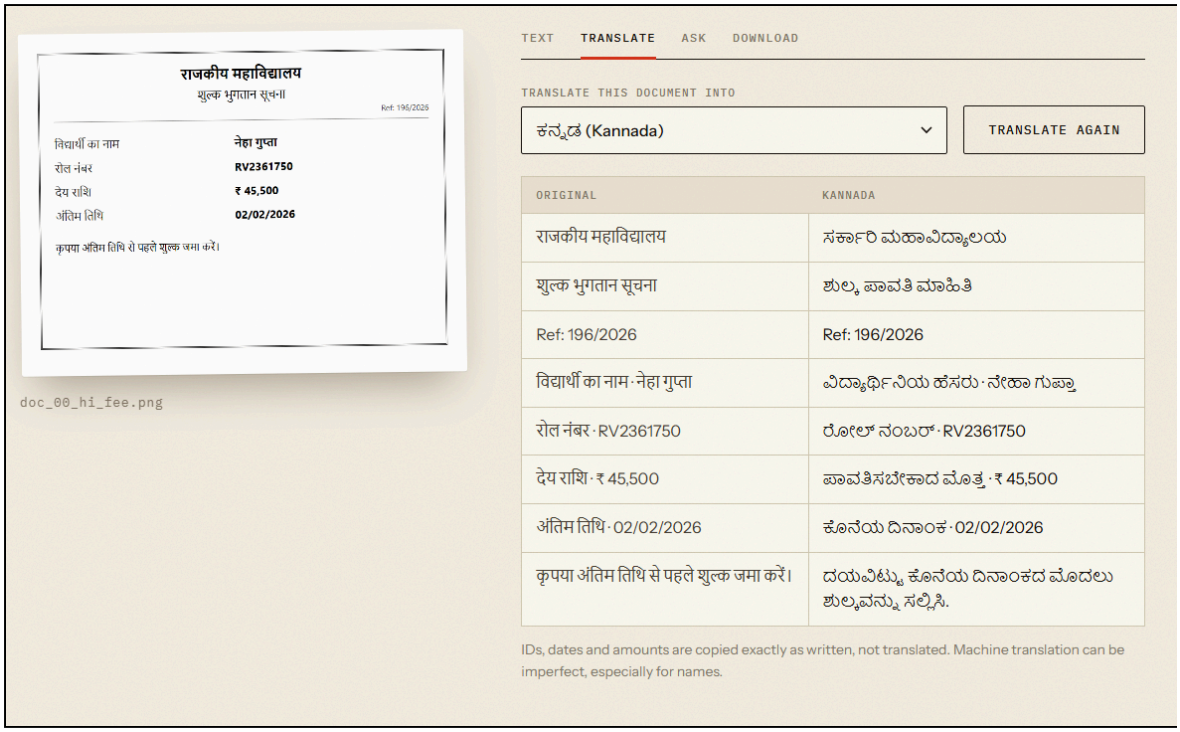


Figure 4.6: The same document translated into Kannada, shown beside the original

4.4.6 Question Answering

Suggested questions and a free-text box allow the user to query the document (Figure 4.7). The answers are drawn only from the document.

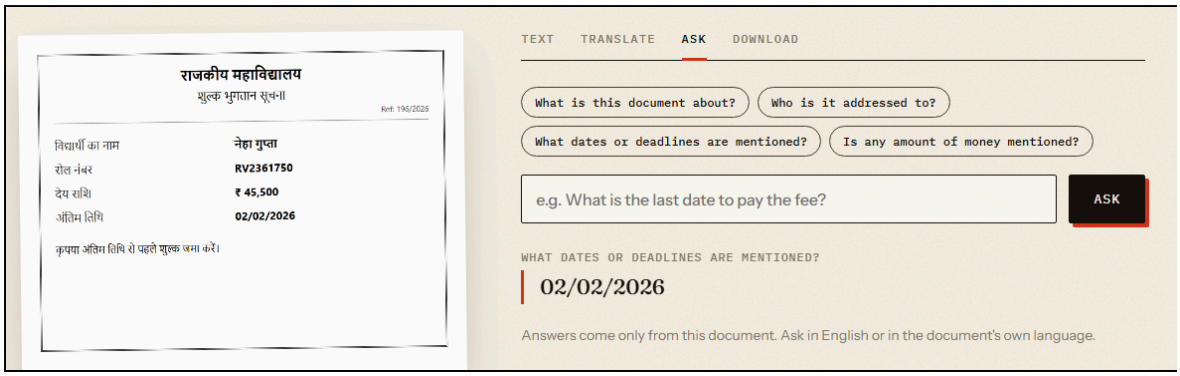


Figure 4.7: Question answering on the document

4.4.7 Mobile Layout

The layout adapts to narrow screens by collapsing to a single column (Figure 4.8). It was tested at a width of 390 pixels.

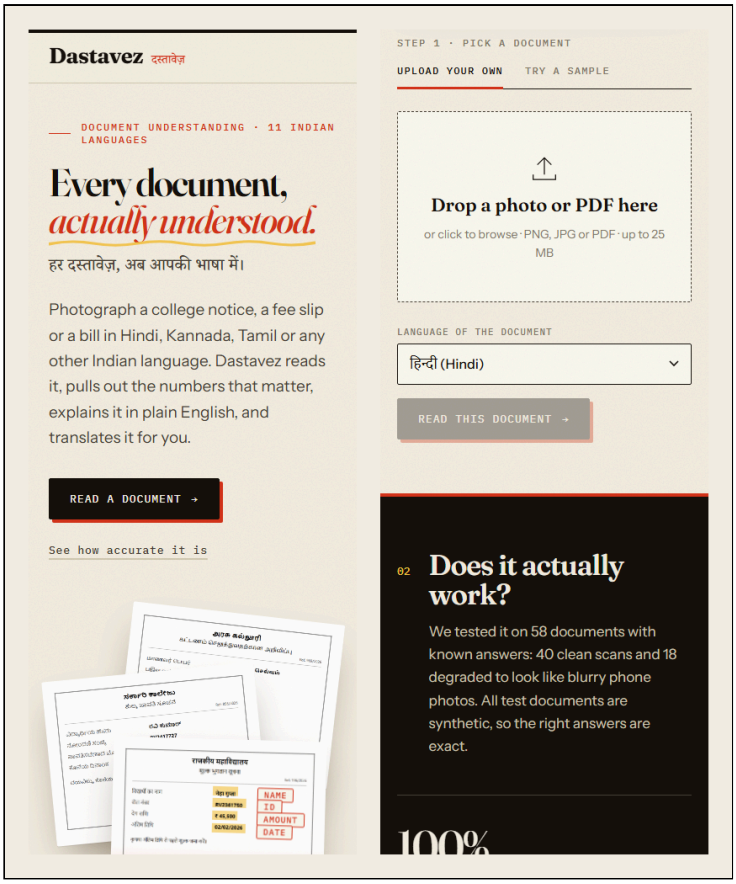


Figure 4.8: Layout on a phone-sized screen

4.4.8 Information Section

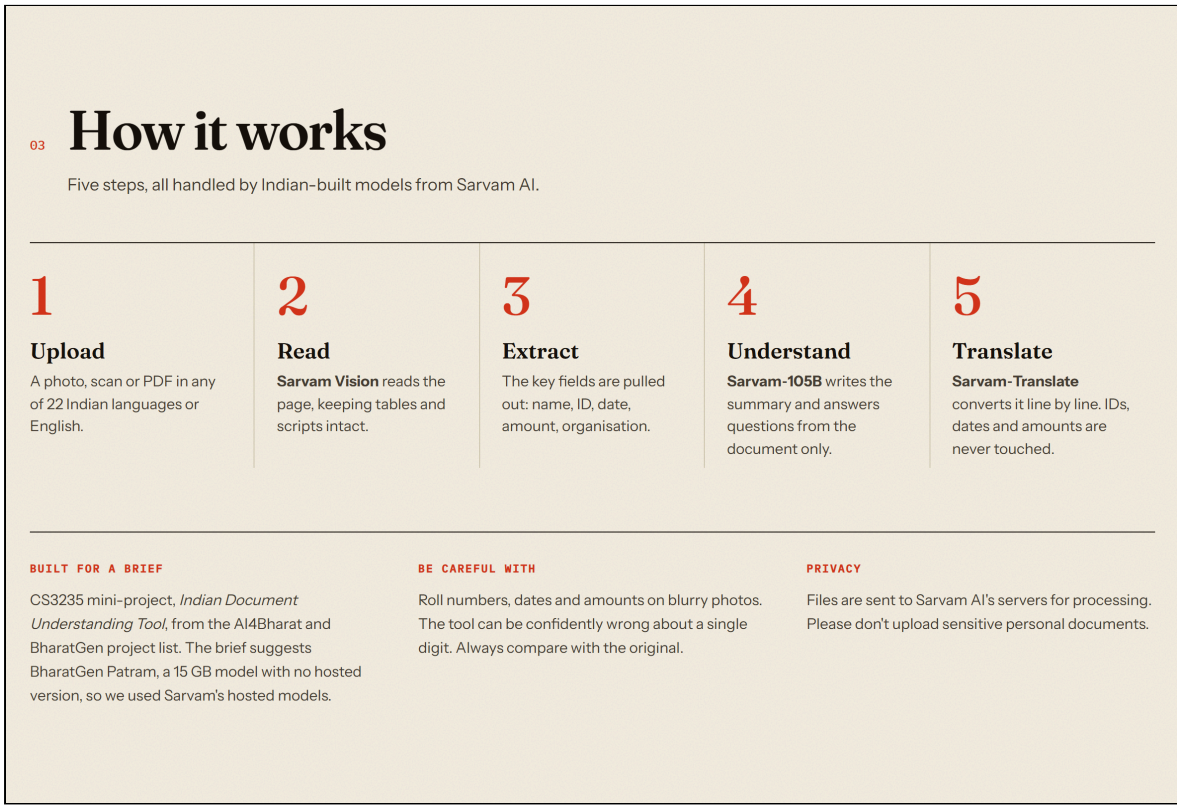


Figure 4.9: The "How it works" section of the website

4.5 Robustness and Security

- Uploaded files are validated for type (PNG, JPEG or PDF), size (25 MB) and language; unknown sample names, including attempts at path traversal, are rejected.
- Failures of the external service are shown as readable messages (for example, "credits used up"), not as technical errors. Network errors and rate limits are retried automatically; browser requests time out after 120 seconds.
- Every finished evaluation document is cached, so an interrupted evaluation resumes where it stopped; documents that fail because of an API error are skipped and reported, and are never scored as wrong answers.
- The API key is read from a local file that is excluded from version control and from the submitted package.

CHAPTER 5
TESTING AND RESULTS

5.1 Test Sets

No public benchmark of Indian-language notices with exact ground truth matched the task, so two test sets were constructed. Every name, number and date in the documents is invented, so the correct answers are known exactly. The documents are rendered from HTML by Microsoft Edge, which shapes Indic scripts correctly.

Table 5.1: Composition of the test sets

Test set	Contents
Clean	40 documents (16 Hindi, 12 Kannada, 12 Tamil) in two layouts (fee-payment notice and examination admit card), with four fields each: 160 fields and 40 questions.
Phone-photograph	18 degraded copies of clean documents, in three severity levels with six documents each (two per language). Tilt up to 2, 4 and 6 degrees; Gaussian blur; resolution reduced to 70%, 50% and 38%; uneven lighting; sensor noise; and JPEG quality 45, 30 and 20. In total 72 fields and 18 questions. Together with the clean set: 58 test cases from 40 unique documents.



Figure 5.1: The same layout at each severity level (Tamil fee notice)

5.2 Evaluation Metrics

- **Field-extraction accuracy:** the fraction of fields that match the ground truth. Dates and amounts are compared by their digits only (so that 15000 equals 15,000); text comparison ignores spaces and punctuation.
- **Question-answering accuracy:** for each document, one question (amount due, last date, examination date or examination city) is answered from the parsed text. The answer is correct if the expected value appears in it.

5.3 Results

Table 5.2: Accuracy on the clean and phone-photograph test sets

Test set	Cases	Field accuracy	Question accuracy
Clean	40	100% (160/160)	100% (40/40)
Phone-photograph, all levels	18	83.3% (60/72)	88.9% (16/18)
Mild	6	100% (24/24)	100% (6/6)
Medium	6	100% (24/24)	100% (6/6)
Harsh	6	50.0% (12/24)	66.7% (4/6)

The result of 100% on the clean set was verified not to be a scoring artefact: no field was empty, and the only non-identical strings were formatting differences such as 15000 and 15,000. Accuracy falls only on the harsh images. By language, the field accuracy on the phone-photograph set was 87.5% for Hindi, 79.2% for Kannada and 83.3% for Tamil, but with only six documents per language these differences are within the range of chance. Figure 5.2 reproduces the corresponding results section of the website.



Figure 5.2: Results section of the website, with accuracy against photograph quality

Table 5.3: Field accuracy by field on the phone-photograph set

Field	Correct / total	Accuracy
Student name	9 / 9	100.0%
Amount due	8 / 9	88.9%
Last date for payment	8 / 9	88.9%
Examination centre (city)	8 / 9	88.9%
Roll number	14 / 18	77.8%
Candidate name	7 / 9	77.8%
Examination date	6 / 9	66.7%

5.4 Failure Analysis

All 12 wrong fields and both wrong answers occurred in documents of the harsh category. The raw output of the models was examined for each. Failure modes 1 to 3 in Table 5.4 together account for all 12 wrong fields (4 + 3 + 5); failure mode 4 overlaps with them.

Table 5.4: Failure modes observed on the harsh phone-photograph documents

No.	Failure mode	Evidence	Fields
1	Single-digit misreads in blurred numbers	Roll number `RV2568829` read as `RVZ168829`; `RV2354833` as `RV2354811`; `RV2589988` as `RV2589968`; amount `32000` as `12,000`. Confusions observed: 3 to 1, 5 to 1, 8 to 6 and 2 to Z.	4
2	Misreading of date digits	`16/07/2026` read as `16/07/2024`; `16/11/2026` as `16/11/2024`; `13/12/2026` as `13/11/2024` (6 read as 4).	3
3	Unreadable labels leading to empty fields	On a Kannada admit card the OCR misread all four labels, so the extractor could not map any field and returned `null` for all four. On a Tamil admit card the candidate name was returned as `null`.	5
4	OCR hallucination	On a Tamil fee notice the parsed text contained a Tirukkural verse, the phrase “Dear mother” and a Kannada organisation name, none of which appear on the page. The separate field reader still returned the name and almost all of the roll number. The question about the last date then returned “Not found”.	overlaps

5.5 Discussion

- **Errors lie in digits, not in words.** Names were either read correctly or left empty; they were never silently misspelt. Roll numbers and dates therefore carry the real risk, and a person should verify them.
- **The model withholds names but guesses digits.** Failure modes 1 and 2 are confident wrong answers, which are more dangerous than an empty field.
- **Question answering is limited by the OCR text.** Both wrong answers resulted from poor parsed text: a wrong city taken from a misread header, and “Not found” because the date was absent from the text. Supplying the extracted fields to the question-answering step as a second source would probably correct the second case. This was deliberately not done, to avoid tuning the system on its own test set.

CHAPTER 6

CHALLENGES ENCOUNTERED AND SOLUTIONS

Most of the engineering effort was spent on problems that appeared only when the system was tested against the real services. Each entry in Table 6.1 was found by experiment.

Table 6.1: Problems encountered and the solutions adopted

No.	Problem	Observation	Solution
1	Models in the brief too large	Patram: 15.3 GB. Available graphics card: 6 GB. Measured download speed: about 0.1 to 0.4 MB/s.	Hosted Indian models from Sarvam AI were used, and the change is documented.
2	Hosted inference blocked	Hugging Face returned “402 Payment Required”; Patram has no hosted endpoint; the AI4Bharat demonstration space returned a configuration error.	Moved to the Sarvam AI API, which offers free starting credits.
3	Model deprecated	`sarvam-m` returned an error requesting the use of `sarvam-105b`.	Model replaced; all chat calls re-tested.
4	Truncated answers	The 105B model is a reasoning model that uses about 1,000 tokens before answering, so a limit of 1,500 tokens cut answers short.	Budget raised to 4,000 tokens, with one retry at double the budget.
5	Corrupted translation	Translating a whole page merged lines, dropped a line, broke table markup, and altered a date in Tamil.	Cell-by-cell translation, protected values and a number check (Section 4.3).
6	Weak translations	`mayura:v1` mistranslated common terms and mixed scripts.	`sarvam-translate:v1` adopted after side-by-side comparison.
7	Slow translation	Call latency varied from 1 to 7 seconds; a document took 22 to 53 seconds.	One parallel worker per cell (1 to 3 seconds); a 120-second time-out prevents the page from waiting indefinitely.
8	Long paragraphs truncated	The API accepts a limited number of characters per call and an early version silently truncated longer lines.	Long lines are split at sentence ends.
9	Parser returned images	On one blurred photograph the OCR returned a region as a base64 image instead of text.	Embedded image blobs are removed from the parsed text.

No.	Problem	Observation	Solution
10	Free credits exhausted	Evaluations stopped part-way with HTTP 402, and the first scoring code counted these failures as wrong answers, giving a misleading 27.8%.	Failures are skipped and reported separately; finished documents are cached; a clear message is displayed.
11	Summary in the wrong language	The summary of a Hindi photograph was returned in Hindi.	Explicit instruction to write the summary in English; re-tested on Hindi, Kannada and Tamil.
12	Assumed gender	A summary used “her” on the basis of a first name.	The prompt now forbids guessing gender.

CHAPTER 7
DEVIATIONS FROM THE PROJECT BRIEF

The deviations from the models and deliverables suggested in the project brief are listed in Table 7.1, with the reason for each.

Table 7.1: Deviations from the project brief

Brief suggested	What was done and why
BharatGen Patram for document vision	Sarvam Vision 2.1 was used. Patram is a 15.3 GB model with no hosted version and could not be run on the available hardware or downloaded on the available connection.
AI4Bharat IndicTrans2 for translation	Sarvam-Translate v1 was used. IndicTrans2 is access-gated and, like Patram, would have required local execution, while hosted inference required paid credits.
A GitHub repository	The complete source code is delivered as a ZIP package. See the README file for setup instructions.

All models used remain Indian-built, and the task, the evaluation metrics and the number of test cases follow the brief. The single module `docai.py` is the only part of the system that communicates with the models, so a local Patram backend could replace it without changes elsewhere.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

An end-to-end Indian Document Understanding Tool was designed, implemented and evaluated. It reads Indian-language documents, extracts their key fields, summarises them in English, translates them into eleven languages and answers questions about them, using only Indian-built models. On 40 clean synthetic documents it made no errors in field extraction or question answering, and on 18 documents degraded to resemble phone photographs it achieved 83.3% field accuracy and 88.9% question-answering accuracy, with all errors confined to the harshest level of degradation. The analysis of these errors showed that the system fails mainly by misreading individual digits with full confidence, which is the main practical risk and the reason that important values should always be checked against the original document.

The work also showed that a considerable part of the effort in building applications on external AI services lies in handling their real behaviour: reasoning-token budgets, model deprecation, rate and credit limits, merged or altered translations, and hallucinated OCR output. The measures adopted in response are described in Chapters 4 and 6.

8.2 Limitations

- The test documents are synthetic and use only two clean layouts; real notices are less regular and may contain stamps, handwriting and photographs of curved paper.
- Only Hindi, Kannada and Tamil were evaluated, although the application accepts eleven languages.
- Each language has only six degraded documents, so differences between languages are not statistically meaningful.
- Question answering is scored by string containment, which can accept a correct value that appears within a wrong sentence.
- The tool depends on a third-party API; when its credits are exhausted, new documents cannot be analysed. Documents are sent to Sarvam AI’s servers, so sensitive personal documents should not be uploaded.
- Machine translation can be imperfect, especially for personal names.

8.3 Future Scope

- Supply the extracted fields to the question-answering step as a second source, to recover from OCR hallucinations.

- Add a digit-confidence check or a second-reader vote for roll numbers, dates and amounts, which are the fields that fail confidently.
- Build a larger test set from real, consented photographs, including handwriting and additional layouts.
- Provide BharatGen Patram and AI4Bharat IndicTrans2 as an optional local backend behind `docai.py`, for privacy and independence from paid services.
- Use Sarvam's document-to-document translation to preserve the layout of translated PDF files.

REFERENCES

1. “Mini Projects using AI4Bharat & BharatGen,” CS3235 project brief, Undergraduate LLM / Multimodal AI Projects, October 2026.
2. Sarvam AI, “Sarvam AI Developer Documentation.” [Online]. Available: https://docs.sarvam.ai. [Accessed: Nov. 2026].
3. Sarvam AI, “Document Intelligence Overview (Sarvam Vision 2.1).” [Online]. Available: https://docs.sarvam.ai/api/api-guides-tutorials/document-intelligence/overview. [Accessed: Nov. 2026].
4. BharatGen, “BharatGen models on Hugging Face.” [Online]. Available: https://huggingface.co/bharatgenai. [Accessed: Nov. 2026].
5. AI4Bharat, “IndicTrans2.” [Online]. Available: https://github.com/AI4Bharat/IndicTrans2. [Accessed: Nov. 2026].
6. Pallets Projects, “Flask Documentation.” [Online]. Available: https://flask.palletsprojects.com. [Accessed: Nov. 2026].
7. Anthropic, “Claude Code.” [Online]. Available: https://claude.com/claude-code. [Accessed: Nov. 2026].

APPENDIX A
INSTALLATION AND EXECUTION

The source package contains a README file with complete instructions. In summary:

```
pip install -r requirements.txt
copy .env.example .env      (then enter the Sarvam API key in .env)
python server.py            (opens http://localhost:5050)
```

On Windows, double-clicking `run_app.bat` performs the same action as `python server.py`. A Sarvam API key can be obtained free of charge at dashboard.sarvam.ai. The test sets and the evaluation can be regenerated and re-run with the following commands.

```
python make_testset.py        # 40 clean test documents (requires
Microsoft Edge)
python make_hard_testset.py   # 18 degraded phone-photograph copies
python evaluate.py            # evaluates the clean set   -> results/
python evaluate.py hard       # evaluates the degraded set ->
results/hard_*
```